

Concatenation of Array

```
class Solution(object):
    def getConcatenation(self, nums):
        return nums + nums
```

Shuffle the Array

```
class Solution(object):
    def shuffle(self, nums, n):
        ans = []
        for i in range(n):
            ans.append(nums[i])           # x1, x2, ...
            ans.append(nums[i+n])         # y1, y2, ...
        return ans
```

Max Consecutive Ones

```
class Solution(object):
    def findMaxConsecutiveOnes(self, nums):
        max_count = 0
        current = 0

        for num in nums:
            if num == 1:
                current += 1
                max_count = max(max_count, current)
            else:
                current = 0

        return max_count
```

Set Mismatch

```
class Solution(object):
    def findErrorNums(self, nums):
        n = len(nums)
        seen = set()
        duplicate = -1

        for num in nums:
            if num in seen:
                duplicate = num
            seen.add(num)

        # sum of numbers from 1..n
        expected_sum = n * (n + 1) // 2
        actual_sum = sum(nums)

        missing = expected_sum - (actual_sum - duplicate)

        return [duplicate, missing]
```

How Many Numbers Are Smaller Than the Current Number

```
class Solution(object):
    def smallerNumbersThanCurrent(self, nums):
        # Sort a copy of nums
        sorted_nums = sorted(nums)

        # Map each number to the first index where it appears
        first_index = {}
        for i, v in enumerate(sorted_nums):
            if v not in first_index:
                first_index[v] = i # number of elements smaller

        # Build result: for each num, fetch its rank
        return [first_index[num] for num in nums]
```

Find All Numbers Disappeared in an Array

```
class Solution(object):
    def findDisappearedNumbers(self, nums):
        for i in range(len(nums)):
            index = abs(nums[i]) - 1
            if nums[index] > 0:
                nums[index] = -nums[index] # mark as visited

        result = []
        for i in range(len(nums)):
            if nums[i] > 0: # not visited
                result.append(i + 1)

        return result
```

Build an Array With Stack Operations

```
class Solution(object):
    def buildArray(self, target, n):
        result = []
        current = 1

        for t in target:
            # Push numbers until we reach t
            while current < t:
                result.append("Push")
                result.append("Pop")
                current += 1

            # Push the target number
            result.append("Push")
            current += 1

        return result
```

Evaluate Reverse Polish Notation

```
class Solution(object):
    def evalRPN(self, tokens):
        stack = []

        for t in tokens:
            if t not in ["+", "-", "*", "/"]:
                stack.append(int(t))
            else:
                b = stack.pop()
                a = stack.pop()

                if t == "+":
                    stack.append(a + b)
                elif t == "-":
                    stack.append(a - b)
                elif t == "*":
                    stack.append(a * b)
                else: # division
                    # Python division needs truncation toward zero
                    stack.append(int(float(a) / b))

        return stack[0]
```

Exclusive Time of Functions

```
class Solution(object):
    def exclusiveTime(self, n, logs):
        res = [0] * n
        stack = []
        prev = 0

        for log in logs:
            f, typ, t = log.split(':')
            f = int(f)
            t = int(t)

            if typ == "start":
                if stack:
                    res[stack[-1]] += t - prev
                stack.append(f)
                prev = t
            else: # "end"
                res[stack.pop()] += t - prev + 1
                prev = t + 1

        return res
```

Final Prices With a Special Discount in a Shop

```
class Solution(object):
```

```

def finalPrices(self, prices):
    stack = []
    n = len(prices)
    res = prices[:]

    for i in range(n):
        while stack and prices[stack[-1]] >= prices[i]:
            idx = stack.pop()
            res[idx] -= prices[i]
        stack.append(i)

    return res

```

Daily Temperature

```

class Solution(object):
    def dailyTemperatures(self, temperatures):
        n = len(temperatures)
        res = [0] * n
        stack = [] # stores indices

        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                idx = stack.pop()
                res[idx] = i - idx
            stack.append(i)

        return res

```

Largest Rectangle in Histogram

```

class Solution(object):
    def largestRectangleArea(self, heights):
        stack = [] # store indices
        heights.append(0) # sentinel to flush stack
        max_area = 0

        for i, h in enumerate(heights):
            while stack and heights[stack[-1]] > h:
                height = heights[stack.pop()]
                width = i if not stack else i - stack[-1] - 1
                max_area = max(max_area, height * width)
            stack.append(i)

        return max_area

```

Number of Students Unable to Eat Lunch.

```

class Solution(object):
    def countStudents(self, students, sandwiches):
        count0 = students.count(0)
        count1 = students.count(1)

```

```

for s in sandwiches:
    if s == 0:
        if count0 == 0:
            return count1 # all remaining are 1's
        count0 -= 1
    else:
        if count1 == 0:
            return count0 # all remaining are 0's
        count1 -= 1

return 0

```

Time Needed to Buy Tickets.

```

class Solution(object):
    def timeRequiredToBuy(self, tickets, k):
        time = 0

        for i in range(len(tickets)):
            if i <= k:
                time += min(tickets[i], tickets[k])
            else:
                time += min(tickets[i], tickets[k] - 1)

        return time

```

Implement Queue using Stacks

```

class MyQueue(object):

    def __init__(self):
        self.inStack = []
        self.outStack = []

    def push(self, x):
        self.inStack.append(x)

    def pop(self):
        self.peak()
        return self.outStack.pop()

    def peak(self):
        if not self.outStack:
            while self.inStack:
                self.outStack.append(self.inStack.pop())
        return self.outStack[-1]

    def empty(self):
        return not self.inStack and not self.outStack

```

Find K Pairs With Smallest Sums

```
import heapq

class Solution(object):
    def kSmallestPairs(self, nums1, nums2, k):
        if not nums1 or not nums2:
            return []

        min_heap = []
        res = []

        # push first column (nums1[i], nums2[0])
        for i in range(min(k, len(nums1))):
            heapq.heappush(min_heap, (nums1[i] + nums2[0], i, 0))

        while min_heap and len(res) < k:
            s, i, j = heapq.heappop(min_heap)
            res.append([nums1[i], nums2[j]])

            # move in nums2: (i, j+1)
            if j + 1 < len(nums2):
                heapq.heappush(min_heap, (nums1[i] + nums2[j+1], i, j+1))

        return res
```

Construct Target Array With Multiple Sums.

```
import heapq

class Solution(object):
    def isPossible(self, target):
        if len(target) == 1:
            return target[0] == 1

        # max-heap using negatives
        total = sum(target)
        target = [-x for x in target]
        heapq.heapify(target)

        while True:
            largest = -heapq.heappop(target)
            rest = total - largest

            if largest == 1 or rest == 1:
                return True

            if rest == 0 or largest < rest:
                return False

            new_val = largest % rest
            if new_val == 0:
```

```
return False
```

```
total = rest + new_val  
heapq.heappush(target, -new_val)
```

Detect Capital.

```
class Solution(object):  
    def detectCapitalUse(self, word):  
        return word.isupper() or word.islower() or word.istitle()
```

Masking Personal Information.

```
class Solution(object):  
    def maskPII(self, s):  
        s = s.strip()  
  
        # Check if it's an email  
        if '@' in s:  
            s = s.lower()  
            name, domain = s.split('@')  
            return name[0] + "*****" + name[-1] + "@" + domain  
  
        # Otherwise it's a phone number  
        digits = "".join([c for c in s if c.isdigit()])  
        local = "****-****-" + digits[-4:]  
        country_len = len(digits) - 10  
  
        if country_len > 0:  
            return "+" + ("*" * country_len) + "-" + local  
        else:  
            return local
```

License Key Formatting

```
class Solution(object):  
    def licenseKeyFormatting(self, s, k):  
        # remove existing dashes and uppercase  
        s = s.replace("-", "").upper()  
  
        n = len(s)  
        if n == 0:  
            return ""  
  
        # first group size  
        first = n % k  
        res = []  
  
        # add first group if exists  
        if first > 0:  
            res.append(s[:first])  
  
        # add the rest in groups of k  
        for i in range(first, n, k):
```

```

        res.append(s[i:i+k])

    return "-".join(res)

```

Repeated Substring Pattern.

```

class Solution(object):
    def repeatedSubstringPattern(self, s):
        return s in (s + s)[1:-1]

```

Repeated String Match.

```

class Solution(object):
    def repeatedStringMatch(self, a, b):
        cnt = 1
        s = a

        # repeat until length >= len(b)
        while len(s) < len(b):
            s += a
            cnt += 1

        # check once
        if b in s:
            return cnt

        # check one more repetition
        s += a
        if b in s:
            return cnt + 1

        return -1

```

Rotate String.

```

class Solution(object):
    def rotateString(self, s, goal):
        return len(s) == len(goal) and goal in (s + s)

```

Remove Duplicates from Sorted List.

```

class Solution(object):
    def deleteDuplicates(self, head):
        curr = head

        while curr and curr.next:
            if curr.val == curr.next.val:
                curr.next = curr.next.next # skip duplicate
            else:
                curr = curr.next          # move forward

        return head

```


Odd Even Linked List

```
class Solution(object):
    def oddEvenList(self, head):
        if not head or not head.next:
            return head

        odd = head
        even = head.next
        even_head = even  # save start of even list

        while even and even.next:
            odd.next = even.next
            odd = odd.next

            even.next = odd.next
            even = even.next

        odd.next = even_head
        return head
```

Implement Trie (Prefix Tree).

```
class Trie(object):

    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node['#'] = True  # end of word

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node:
                return False
            node = node[ch]
        return '#' in node

    def startsWith(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node:
                return False
            node = node[ch]
        return True
```

Two Sum

```
class Solution(object):
    def twoSum(self, nums, target):
        seen = {} # value → index

        for i, num in enumerate(nums):
            diff = target - num
            if diff in seen:
                return [seen[diff], i]
            seen[num] = i
```

Copy List with Random Pointer.

```
class Solution(object):
    def copyRandomList(self, head):
        if not head:
            return None

        # 1 Insert copied nodes after each original node
        curr = head
        while curr:
            newNode = Node(curr.val, curr.next)
            curr.next = newNode
            curr = newNode.next

        # 2 Assign random pointers for the copied nodes
        curr = head
        while curr:
            if curr.random:
                curr.next.random = curr.random.next
            curr = curr.next.next

        # 3 Separate the two lists
        curr = head
        newHead = head.next
        copy = newHead

        while curr:
            curr.next = curr.next.next
            if copy.next:
                copy.next = copy.next.next
            curr = curr.next
            copy = copy.next

        return newHead
```

First Missing Positive

```
class Solution(object):
    def firstMissingPositive(self, nums):
        n = len(nums)

        # 1. Replace all out-of-range numbers with a dummy value (> n)
```

```

for i in range(n):
    if nums[i] <= 0 or nums[i] > n:
        nums[i] = n + 1

# 2. Mark existing numbers by negating the index position
for i in range(n):
    val = abs(nums[i])
    if val <= n:
        nums[val - 1] = -abs(nums[val - 1])

# 3. First positive index + 1 is our answer
for i in range(n):
    if nums[i] > 0:
        return i + 1

# If all indices are marked, answer is n+1
return n + 1

```

Find the Highest Altitude.

```

class Solution(object):
    def largestAltitude(self, gain):
        altitude = 0
        highest = 0

        for g in gain:
            altitude += g
            highest = max(highest, altitude)

        return highest

```

Make Sum Divisible by P

```

class Solution(object):
    def minSubarray(self, nums, p):
        need = sum(nums) % p
        if need == 0:
            return 0

        prefix = 0
        seen = {0: -1}
        res = len(nums)

        for i, x in enumerate(nums):
            prefix = (prefix + x) % p
            target = (prefix - need) % p

            if target in seen:
                res = min(res, i - seen[target])

            seen[prefix] = i

        return res if res < len(nums) else -1

```

ways to make a Fair array

```
class Solution(object):
    def waysToMakeFair(self, nums):
        n = len(nums)

        # prefix sums for even and odd indices
        preE = [0] * (n + 1)
        preO = [0] * (n + 1)

        for i in range(n):
            preE[i+1] = preE[i]
            preO[i+1] = preO[i]
            if i % 2 == 0:
                preE[i+1] += nums[i]
            else:
                preO[i+1] += nums[i]

        res = 0

        for i in range(n):
            # remove nums[i]
            # left evens = preE[i]
            # left odds = preO[i]

            # after removal, right side indices shift (even ↔ odd)
            right_even = preO[n] - preO[i+1] # originally odd → now even
            right_odd = preE[n] - preE[i+1] # originally even → now odd

            if preE[i] + right_even == preO[i] + right_odd:
                res += 1

        return res
```

Minimum Absolute Difference.

```
class Solution(object):
    def minimumAbsDifference(self, arr):
        arr.sort()
        minDiff = float('inf')
        res = []

        for i in range(1, len(arr)):
            diff = arr[i] - arr[i-1]
            if diff < minDiff:
                minDiff = diff
                res = [[arr[i-1], arr[i]]]
            elif diff == minDiff:
                res.append([arr[i-1], arr[i]])

        return res
```

Reduction Operations to Make the Array Elements Equal.

```
class Solution(object):
    def reductionOperations(self, nums):
        nums.sort()
        res = 0
        steps = 0

        for i in range(1, len(nums)):
            if nums[i] != nums[i-1]:
                steps += 1
            res += steps

        return res
```

Merge Intervals.

```
class Solution(object):
    def merge(self, intervals):
        intervals.sort()
        res = []

        for start, end in intervals:
            if not res or start > res[-1][1]:
                res.append([start, end]) # no overlap
            else:
                res[-1][1] = max(res[-1][1], end) # merge overlap

        return res
```

Sort an Array

```
class Solution(object):
    def sortArray(self, nums):
        return self.mergeSort(nums)

    def mergeSort(self, nums):
        if len(nums) <= 1:
            return nums

        mid = len(nums) // 2
        left = self.mergeSort(nums[:mid])
        right = self.mergeSort(nums[mid:])

        return self.merge(left, right)

    def merge(self, left, right):
        res = []
        i = j = 0

        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                res.append(left[i])
                i += 1
            else:
```

```

        res.append(right[j])
        j += 1

    res.extend(left[i:])
    res.extend(right[j:])
    return res

```

Insertion Sort List

```

class Solution(object):
    def insertionSortList(self, head):
        if not head:
            return head

        dummy = ListNode(0)
        curr = head

        while curr:
            prev = dummy

            # find position to insert curr
            while prev.next and prev.next.val < curr.val:
                prev = prev.next

            nextNode = curr.next
            curr.next = prev.next
            prev.next = curr
            curr = nextNode

        return dummy.next

```

Peak Index in a Mountain Array.

```

class Solution(object):
    def peakIndexInMountainArray(self, arr):
        l, r = 0, len(arr) - 1

        while l < r:
            mid = (l + r) // 2

            if arr[mid] < arr[mid + 1]:
                l = mid + 1    # peak is on the right
            else:
                r = mid        # peak is here or on the left

        return l

```

Sum of Square Numbers.

```

class Solution(object):
    def judgeSquareSum(self, c):
        left, right = 0, int(c**0.5)

```

```

while left <= right:
    s = left*left + right*right
    if s == c:
        return True
    elif s < c:
        left += 1
    else:
        right -= 1

return False

```

Search in Rotated Sorted Array

```

class Solution(object):
    def search(self, nums, target):
        l, r = 0, len(nums) - 1

        while l <= r:
            mid = (l + r) // 2

            if nums[mid] == target:
                return mid

            # left half sorted
            if nums[l] <= nums[mid]:
                if nums[l] <= target < nums[mid]:
                    r = mid - 1
                else:
                    l = mid + 1

            # right half sorted
            else:
                if nums[mid] < target <= nums[r]:
                    l = mid + 1
                else:
                    r = mid - 1

        return -1

```

Binary Search

```

class Solution(object):
    def search(self, nums, target):
        l, r = 0, len(nums) - 1

        while l <= r:
            mid = (l + r) // 2

            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                l = mid + 1
            else:

```

```
r = mid - 1
```

```
return -1
```

Beautiful Array.

```
class Solution(object):
    def beautifulArray(self, n):
        res = [1]

        while len(res) < n:
            odd = [x * 2 - 1 for x in res]
            even = [x * 2 for x in res]
            res = odd + even

        return [x for x in res if x <= n]
```

Reverse Pairs.

```
class Solution(object):
    def reversePairs(self, nums):
        return self.mergeSort(nums, 0, len(nums)-1)

    def mergeSort(self, nums, l, r):
        if l >= r:
            return 0

        mid = (l + r) // 2
        count = self.mergeSort(nums, l, mid) + self.mergeSort(nums, mid+1, r)

        # Count reverse pairs
        j = mid + 1
        for i in range(l, mid+1):
            while j <= r and nums[i] > 2 * nums[j]:
                j += 1
            count += j - (mid + 1)

        # Merge the two sorted halves
        temp = []
        i, j = l, mid + 1

        while i <= mid and j <= r:
            if nums[i] <= nums[j]:
                temp.append(nums[i])
                i += 1
            else:
                temp.append(nums[j])
                j += 1

        while i <= mid:
            temp.append(nums[i])
            i += 1

        while j <= r:
```



```
temp.append(nums[j])
j += 1
```

```
nums[l:r+1] = temp
return count
```

Construct Binary Tree from Inorder and Postorder Traversal.

```
class Solution(object):
    def buildTree(self, inorder, postorder):
        if not inorder or not postorder:
            return None

        # Map value → index in inorder for O(1) lookup
        idx = {v: i for i, v in enumerate(inorder)}

        def build(l, r):
            if l > r:
                return None

            # Last element in postorder is the root
            root_val = postorder.pop()
            root = TreeNode(root_val)

            # In inorder, elements right of root_val belong to right subtree
            mid = idx[root_val]

            # Build right subtree *before* left because of postorder pop()
            root.right = build(mid + 1, r)
            root.left = build(l, mid - 1)

            return root

        return build(0, len(inorder) - 1)
```

Linked List Cycle

```
class Solution(object):
    def hasCycle(self, head):
        slow = fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        if slow == fast:
            return True

        return False
```

3Sum Closest.

```
class Solution(object):
    def threeSumClosest(self, nums, target):
        nums.sort()
        closest = float('inf')

        for i in range(len(nums) - 2):
            l, r = i + 1, len(nums) - 1

            while l < r:
                s = nums[i] + nums[l] + nums[r]

                # Update closest if this sum is closer
                if abs(s - target) < abs(closest - target):
                    closest = s

                if s == target:
                    return s
                elif s < target:
                    l += 1
                else:
                    r -= 1

        return closest
```

Magical String

```
class Solution(object):
    def magicalString(self, n):
        if n == 0:
            return 0
        if n <= 3:
            return 1 # magical string starts with "122"

        s = [1, 2, 2]
        i = 2 # pointer to read how many times to append
        num = 1 # next number to append (alternate between 1 and 2)

        while len(s) < n:
            count = s[i] # how many times to append num
            s.extend([num] * count)
            num = 3 - num # swap: 1 -> 2, 2 -> 1
            i += 1

        return s[:n].count(1)
```

Contains Duplicate II

```
class Solution(object):
    def containsNearbyDuplicate(self, nums, k):
        last = {}
```

```

for i, num in enumerate(nums):
    if num in last and i - last[num] <= k:
        return True
    last[num] = i

return False

```

Number of Substrings Containing All Three Characters

```

class Solution(object):
    def numberOfSubstrings(self, s):
        count = {'a': 0, 'b': 0, 'c': 0}
        left = 0
        res = 0

        for right in range(len(s)):
            count[s[right]] += 1

            # shrink window while contains all 'a', 'b', 'c'
            while count['a'] > 0 and count['b'] > 0 and count['c'] > 0:
                res += len(s) - right
                count[s[left]] -= 1
                left += 1

        return res

```

Longest Repeating Character Replacement.

```

class Solution(object):
    def characterReplacement(self, s, k):
        count = {}
        left = 0
        maxFreq = 0
        res = 0

        for right in range(len(s)):
            count[s[right]] = count.get(s[right], 0) + 1
            maxFreq = max(maxFreq, count[s[right]])

            # if replacements needed > k → shrink window
            while (right - left + 1) - maxFreq > k:
                count[s[left]] -= 1
                left -= 1
                left += 2 # quick fix: simply do left += 1 instead of left -=1 then +=2

            # correct window shrink:
            # while (right - left + 1) - maxFreq > k:
            #     count[s[left]] -= 1
            #     left += 1

            # window size is a valid answer
            res = max(res, right - left + 1)

        return res

```

Shortest Palindrome using the KMP prefix function

```
class Solution(object):
    def shortestPalindrome(self, s):
        rev = s[::-1]
        combined = s + "#" + rev

        # Build LPS array (longest prefix suffix)
        lps = [0] * len(combined)
        j = 0

        for i in range(1, len(combined)):
            while j > 0 and combined[i] != combined[j]:
                j = lps[j - 1]
            if combined[i] == combined[j]:
                j += 1
                lps[i] = j

        # lps[-1] = longest palindromic prefix size
        longest = lps[-1]

        # Add reverse of the rest in front
        return rev[:len(s) - longest] + s
```

Longest Happy Prefix

```
class Solution(object):
    def longestPrefix(self, s):
        n = len(s)
        lps = [0] * n
        j = 0

        for i in range(1, n):
            while j > 0 and s[i] != s[j]:
                j = lps[j - 1]
            if s[i] == s[j]:
                j += 1
                lps[i] = j

        # lps[-1] gives the length of longest prefix which is also suffix
        return s[:lps[-1]]
```

Sum of Scores of Built Strings

```
class Solution(object):
    def sumScores(self, s):
        n = len(s)
        Z = [0] * n

        # Z-algorithm
        l = r = 0
        for i in range(1, n):
```

```

    if i <= r:
        Z[i] = min(r - i + 1, Z[i - 1])
    while i + Z[i] < n and s[Z[i]] == s[i + Z[i]]:
        Z[i] += 1
    if i + Z[i] - 1 > r:
        l = i
        r = i + Z[i] - 1

# Sum of scores = sum(Z) + n (full match at index 0)
return sum(Z) + n

```

Merge Two Sorted Lists.

```

class Solution(object):
    def mergeTwoLists(self, list1, list2):
        dummy = ListNode(0)
        curr = dummy

        while list1 and list2:
            if list1.val <= list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next

        # Attach whatever is left
        curr.next = list1 if list1 else list2

        return dummy.next

```

Decode String.

```

class Solution(object):
    def decodeString(self, s):
        num = 0
        curr = ""
        stack = []

        for ch in s:
            if ch.isdigit():
                num = num * 10 + int(ch)    # build full number
            elif ch == "[":
                stack.append((curr, num))    # push current string & number
                curr = ""                    # reset current string
                num = 0
            elif ch == "]":
                prev, k = stack.pop()        # pop previous string & repeat count
                curr = prev + curr * k       # expand
            else:
                curr += ch                    # normal letters go directly

        return curr

```

Find Kth Bit in Nth Binary String.

```
class Solution(object):
    def findKthBit(self, n, k):
        # length of Sn is 2^n - 1
        length = (1 << n) - 1

        # recursive helper
        def solve(n, k, length):
            if n == 1:
                return "0"

            mid = (length // 2) + 1

            if k == mid:          # middle bit is always '1'
                return "1"
            elif k < mid:         # in S_(n-1)
                return solve(n - 1, k, length // 2)
            else:
                # reflected and inverted bit
                k_mirror = mid - (k - mid)
                bit = solve(n - 1, k_mirror, length // 2)
                return "1" if bit == "0" else "0"

        return solve(n, k, length)
```

Combinations.

```
class Solution(object):
    def combine(self, n, k):
        res = []
        curr = []

        def backtrack(start):
            if len(curr) == k:
                res.append(list(curr))
                return

            for i in range(start, n + 1):
                curr.append(i)
                backtrack(i + 1)
                curr.pop()

        backtrack(1)
        return res
```

The k-th Lexicographical Happy String of Length n.

```
class Solution(object):
    def getHappyString(self, n, k):
        chars = ["a", "b", "c"]
        res = []
```

```

def backtrack(curr):
    if len(res) == k:
        return
    if len(curr) == n:
        res.append(curr)
        return

    for ch in chars:
        if not curr or curr[-1] != ch:
            backtrack(curr + ch)

backtrack("")
return res[k-1] if k <= len(res) else ""

```

Restore IP Addresses.

```

class Solution(object):
    def restoreIpAddresses(self, s):
        res = []

        def backtrack(i, parts, curr):
            # If we have 4 parts and used up all chars
            if parts == 4 and i == len(s):
                res.append(curr[:-1]) # remove trailing dot
                return

            # If too many parts or no chars left but need more parts
            if parts == 4 or i == len(s):
                return

            # Try segments of length 1 to 3
            for l in range(1, 4):
                if i + l > len(s):
                    break
                part = s[i:i+l]

                # No leading zero, unless part == "0"
                if (part[0] == '0' and l > 1):
                    continue

                # Value must be <= 255
                if int(part) > 255:
                    continue

                backtrack(i + l, parts + 1, curr + part + ".")

        backtrack(0, 0, "")
        return res

```

Longest Consecutive Sequence

```
class Solution(object):
    def longestConsecutive(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        if not nums:
            return 0

        s = set(nums)
        best = 0

        for x in s:
            # only start counting when x is the beginning of a sequence
            if x - 1 not in s:
                length = 1
                cur = x + 1
                while cur in s:
                    length += 1
                    cur += 1
                if length > best:
                    best = length

        return best
```

Largest Component Size by Common Factor

```
class Solution(object):
    def largestComponentSize(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        from collections import defaultdict
        import math

        # ---- DSU ----
        n = max(nums) + 1
        parent = list(range(n))
        size = [1] * n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        def union(a, b):
            pa, pb = find(a), find(b)
            if pa != pb:
                if size[pa] < size[pb]:
```



```

        pa, pb = pb, pa
        parent[pb] = pa
        size[pa] += size[pb]

# ---- Prime factorization ----
def prime_factors(x):
    pf = set()
    d = 2
    while d * d <= x:
        if x % d == 0:
            pf.add(d)
            while x % d == 0:
                x //= d
            d += 1
    if x > 1:
        pf.add(x)
    return pf

factor_map = {}

# Union numbers using their prime factors
for num in nums:
    factors = prime_factors(num)
    for f in factors:
        if f in factor_map:
            union(num, factor_map[f])
        else:
            factor_map[f] = num

# Count component sizes
comp = defaultdict(int)
ans = 1
for num in nums:
    root = find(num)
    comp[root] += 1
    ans = max(ans, comp[root])

return ans

```

Process Restricted Friend Requests

```

class Solution(object):
    def friendRequests(self, n, restrictions, requests):
        parent = list(range(n))
        size = [1] * n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

```

```

def union(a, b):
    pa, pb = find(a), find(b)
    if pa != pb:
        if size[pa] < size[pb]:
            pa, pb = pb, pa
        parent[pb] = pa
        size[pa] += size[pb]

ans = []

for u, v in requests:
    pu, pv = find(u), find(v)
    ok = True

    for a, b in restrictions:
        pa, pb = find(a), find(b)
        if (pa == pu and pb == pv) or (pa == pv and pb == pu):
            ok = False
            break

    if ok:
        union(pu, pv)
        ans.append(True)
    else:
        ans.append(False)

return ans

```

Minimum Number of Vertices to Reach All Nodes

```

class Solution(object):
    def findSmallestSetOfVertices(self, n, edges):
        indegree = [0] * n

        for a, b in edges:
            indegree[b] += 1

        return [i for i in range(n) if indegree[i] == 0]

```

Add Edges to Make Degrees of All Nodes Even

```

class Solution(object):
    def isPossible(self, n, edges):
        g = [set() for _ in range(n)]
        deg = [0] * n

        for a, b in edges:
            a -= 1
            b -= 1
            g[a].add(b)
            g[b].add(a)
            deg[a] += 1

```

```

deg[b] += 1

odd = [i for i in range(n) if deg[i] % 2 == 1]

if len(odd) == 0:
    return True
if len(odd) == 2:
    a, b = odd
    if b not in g[a]:
        return True
    for i in range(n):
        if i != a and i != b and i not in g[a] and i not in g[b]:
            return True
    return False

if len(odd) == 4:
    a, b, c, d = odd
    if (b not in g[a] and d not in g[c]) or \
       (c not in g[a] and d not in g[b]) or \
       (d not in g[a] and c not in g[b]):
        return True
    return False

return False

```

Maximum Path Quality of a Graph

```

class Solution(object):
    def maximalPathQuality(self, values, edges, maxTime):
        import heapq
        n = len(values)
        g = [[] for _ in range(n)]
        for u, v, t in edges:
            g[u].append((v, t))
            g[v].append((u, t))

        dist = [10**18] * n
        dist[0] = 0
        pq = [(0, 0)]
        while pq:
            d, u = heapq.heappop(pq)
            if d > dist[u]:
                continue
            for v, t in g[u]:
                nd = d + t
                if nd < dist[v]:
                    dist[v] = nd
                    heapq.heappush(pq, (nd, v))

        used = [0] * n
        ans = [0] # use list to avoid nonlocal (works in Python2/3)

```

```

def dfs(u, time, score):
    if used[u] == 0:
        score += values[u]
        used[u] += 1

    if u == 0:
        if score > ans[0]:
            ans[0] = score

    for v, t in g[u]:
        nt = time + t
        if nt <= maxTime and nt + dist[v] <= maxTime:
            dfs(v, nt, score)

    used[u] -= 1

dfs(0, 0, 0)
return ans[0]

```

Find if Path Exists in Graph

```

class Solution(object):
    def validPath(self, n, edges, source, destination):
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)

        seen = [False] * n
        stack = [source]
        seen[source] = True

        while stack:
            u = stack.pop()
            if u == destination:
                return True
            for v in g[u]:
                if not seen[v]:
                    seen[v] = True
                    stack.append(v)

        return False

```

All Paths From Source to Target

```

class Solution(object):
    def allPathsSourceTarget(self, graph):
        n = len(graph)
        ans = []

        def dfs(u, path):
            if u == n - 1:
                ans.append(path[:])

```

```

        return
    for v in graph[u]:
        dfs(v, path + [v])

dfs(0, [0])
return ans

```

Count the Number of Complete Components

```

class Solution(object):
    def countCompleteComponents(self, n, edges):
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)

        seen = [False] * n

        def dfs(u, comp):
            stack = [u]
            seen[u] = True
            while stack:
                x = stack.pop()
                comp.append(x)
                for v in g[x]:
                    if not seen[v]:
                        seen[v] = True
                        stack.append(v)

        ans = 0
        for i in range(n):
            if not seen[i]:
                comp = []
                dfs(i, comp)
                k = len(comp)
                edge_count = 0
                for x in comp:
                    edge_count += len(g[x])
                edge_count //= 2
                if edge_count == k * (k - 1) // 2:
                    ans += 1

        return ans

```

Minimum Score of a Path Between Two Cities

```

class Solution(object):
    def minScore(self, n, roads):
        g = [[] for _ in range(n+1)]
        for a, b, w in roads:
            g[a].append((b, w))
            g[b].append((a, w))

```

```
seen = [False] * (n+1)
stack = [1]
seen[1] = True
ans = 10**9
```

```
while stack:
    u = stack.pop()
    for v, w in g[u]:
        ans = min(ans, w)
        if not seen[v]:
            seen[v] = True
            stack.append(v)
```

```
return ans
```

Island Perimeter

```
class Solution(object):
```

```
    def islandPerimeter(self, grid):
```

```
        rows = len(grid)
```

```
        cols = len(grid[0])
```

```
        peri = 0
```

```
        for r in range(rows):
```

```
            for c in range(cols):
```

```
                if grid[r][c] == 1:
```

```
                    peri += 4
```

```
                    if r > 0 and grid[r-1][c] == 1:
```

```
                        peri -= 2
```

```
                    if c > 0 and grid[r][c-1] == 1:
```

```
                        peri -= 2
```

```
        return peri
```

Map of Highest Peak

```
class Solution(object):
```

```
    def highestPeak(self, isWater):
```

```
        from collections import deque
```

```
        m, n = len(isWater), len(isWater[0])
```

```

ans = [[-1] * n for _ in range(m)]
q = deque()

for i in range(m):
    for j in range(n):
        if isWater[i][j] == 1:
            ans[i][j] = 0
            q.append((i, j))

dirs = [(1,0),(-1,0),(0,1),(0,-1)]

while q:
    x, y = q.popleft()
    for dx, dy in dirs:
        nx, ny = x + dx, y + dy
        if 0 <= nx < m and 0 <= ny < n and ans[nx][ny] == -1:
            ans[nx][ny] = ans[x][y] + 1
            q.append((nx, ny))

return ans

```

Watched Videos by Your Friends

```

class Solution(object):
    def watchedVideosByFriends(self, watchedVideos, friends, id, level):
        from collections import deque, Counter

        n = len(friends)
        seen = [False] * n
        q = deque([id])
        seen[id] = True

        cur_level = 0
        while q and cur_level < level:

```

```

for _ in range(len(q)):
    u = q.popleft()
    for v in friends[u]:
        if not seen[v]:
            seen[v] = True
            q.append(v)
    cur_level += 1

freq = Counter()
while q:
    friend = q.popleft()
    for video in watchedVideos[friend]:
        freq[video] += 1

return sorted(freq.keys(), key=lambda x: (freq[x], x))

```

Shortest Path With Alternating Colors

```

class Solution(object):
    def shortestAlternatingPaths(self, n, redEdges, blueEdges):
        from collections import deque

        red = [[] for _ in range(n)]
        blue = [[] for _ in range(n)]

        for a, b in redEdges:
            red[a].append(b)
        for a, b in blueEdges:
            blue[a].append(b)

        dist = [[10**9] * 2 for _ in range(n)]
        dist[0][0] = 0
        dist[0][1] = 0

```



```

q = deque()
q.append((0, 0)) # red last
q.append((0, 1)) # blue last

while q:
    node, color = q.popleft()
    d = dist[node][color]

    if color == 0:
        for nxt in blue[node]:
            if d + 1 < dist[nxt][1]:
                dist[nxt][1] = d + 1
                q.append((nxt, 1))
    else:
        for nxt in red[node]:
            if d + 1 < dist[nxt][0]:
                dist[nxt][0] = d + 1
                q.append((nxt, 0))

ans = []
for r, b in dist:
    m = min(r, b)
    ans.append(m if m < 10**9 else -1)

return ans

```

Network Delay Time

```

class Solution(object):
    def networkDelayTime(self, times, n, k):
        import heapq

```

```

g = [[] for _ in range(n+1)]
for u, v, w in times:
    g[u].append((v, w))

dist = [10**9] * (n+1)
dist[k] = 0

pq = [(0, k)]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
    for v, w in g[u]:
        nd = d + w
        if nd < dist[v]:
            dist[v] = nd
            heapq.heappush(pq, (nd, v))

ans = max(dist[1:])
return ans if ans < 10**9 else -1

```

[Minimum Weighted Subgraph With the Required Paths](#)

```

class Solution(object):
    def minimumWeight(self, n, edges, src1, src2, dest):
        import heapq

        g = [[] for _ in range(n)]
        rg = [[] for _ in range(n)]

        for a, b, w in edges:
            g[a].append((b, w))

```

```
rg[b].append((a, w))
```

```
def dijkstra(start, graph):
```

```
    dist = [10**18] * n
```

```
    dist[start] = 0
```

```
    pq = [(0, start)]
```

```
    while pq:
```

```
        d, u = heapq.heappop(pq)
```

```
        if d > dist[u]:
```

```
            continue
```

```
        for v, w in graph[u]:
```

```
            nd = d + w
```

```
            if nd < dist[v]:
```

```
                dist[v] = nd
```

```
                heapq.heappush(pq, (nd, v))
```

```
    return dist
```

```
d1 = dijkstra(src1, g)
```

```
d2 = dijkstra(src2, g)
```

```
dr = dijkstra(dest, rg)
```

```
ans = 10**18
```

```
for i in range(n):
```

```
    if d1[i] < 10**18 and d2[i] < 10**18 and dr[i] < 10**18:
```

```
        ans = min(ans, d1[i] + d2[i] + dr[i])
```

```
return ans if ans < 10**18 else -1
```

Design Graph With Shortest Path Calculator

```
class Graph(object):
```

```
    def __init__(self, n, edges):
```

```
        import heapq
```

```
self.n = n
self.g = [[] for _ in range(n)]
for u, v, w in edges:
    self.g[u].append((v, w))
```

```
def addEdge(self, edge):
    u, v, w = edge
    self.g[u].append((v, w))
```

```
def shortestPath(self, node1, node2):
    import heapq
    dist = [10**15] * self.n
    dist[node1] = 0
    pq = [(0, node1)]
```

```
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
    if u == node2:
        return d
    for v, w in self.g[u]:
        nd = d + w
        if nd < dist[v]:
```

```
class Solution(object):
    def findTheCity(self, n, edges, distanceThreshold):
        INF = 10**9
        dist = [[INF] * n for _ in range(n)]

        for i in range(n):
            dist[i][i] = 0
```

```

for u, v, w in edges:
    dist[u][v] = w
    dist[v][u] = w

# Floyd–Warshall
for k in range(n):
    for i in range(n):
        for j in range(n):
            nd = dist[i][k] + dist[k][j]
            if nd < dist[i][j]:
                dist[i][j] = nd

best = 10**9
ans = -1

# Count reachable neighbors
for i in range(n):
    cnt = 0
    for j in range(n):
        if i != j and dist[i][j] <= distanceThreshold:
            cnt += 1

# smallest count → if tie, choose the LARGER city index
if cnt <= best:
    best = cnt
    ans = i

return ans, dist[v] = nd
    heapq.heappush(pq, (nd, v))

return -1

```

Find the City With the Smallest Number of Neighbors at a Threshold Distance

```
class Solution(object):
    def findTheCity(self, n, edges, distanceThreshold):
        INF = 10**9
        dist = [[INF] * n for _ in range(n)]

        for i in range(n):
            dist[i][i] = 0

        for u, v, w in edges:
            dist[u][v] = w
            dist[v][u] = w

        # Floyd–Warshall
        for k in range(n):
            for i in range(n):
                for j in range(n):
                    nd = dist[i][k] + dist[k][j]
                    if nd < dist[i][j]:
                        dist[i][j] = nd

        best = 10**9
        ans = -1

        # Count reachable neighbors
        for i in range(n):
            cnt = 0
            for j in range(n):
                if i != j and dist[i][j] <= distanceThreshold:
                    cnt += 1

            # smallest count → if tie, choose the LARGER city index
            if cnt <= best:
                best = cnt
                ans = i

        return ans
```

[Root Equals Sum of Children](#)

```
class Solution(object):
    def checkTree(self, root):
        return root.val == root.left.val + root.right.val
```

Create Binary Tree From Descriptions

```
class Solution(object):
    def createBinaryTree(self, descriptions):
        from collections import defaultdict

        nodes = {}
        child_set = set()
```

```

for p, c, isLeft in descriptions:
    if p not in nodes:
        nodes[p] = TreeNode(p)
    if c not in nodes:
        nodes[c] = TreeNode(c)

    if isLeft == 1:
        nodes[p].left = nodes[c]
    else:
        nodes[p].right = nodes[c]

    child_set.add(c)

for p, c, _ in descriptions:
    if p not in child_set:
        return nodes[p]

```

Longest Univalue Path

```

class Solution(object):
    def longestUnivaluePath(self, root):
        self.ans = 0

        def dfs(node):
            if not node:
                return 0

            left = dfs(node.left)
            right = dfs(node.right)

            l = r = 0
            if node.left and node.left.val == node.val:
                l = left + 1
            if node.right and node.right.val == node.val:
                r = right + 1

            self.ans = max(self.ans, l + r)
            return max(l, r)

        dfs(root)
        return self.ans

```

Search in a Binary Search Tree

```

class Solution(object):
    def searchBST(self, root, val):
        while root:
            if root.val == val:
                return root
            if val < root.val:
                root = root.left

```

```
    else:
        root = root.right
    return None
```

Insert Into a Binary Search Tree

```
class Solution(object):
    def insertIntoBST(self, root, val):
        if not root:
            return TreeNode(val)

        cur = root
        while True:
            if val < cur.val:
                if cur.left:
                    cur = cur.left
                else:
                    cur.left = TreeNode(val)
                    break
            else:
                if cur.right:
                    cur = cur.right
                else:
                    cur.right = TreeNode(val)
                    break

        return root
```

Delete node in BST

```
class Solution(object):
    def deleteNode(self, root, key):

        if not root:
            return None

        if key < root.val:
            root.left = self.deleteNode(root.left, key)
        elif key > root.val:
            root.right = self.deleteNode(root.right, key)
        else:
            if not root.left:
                return root.right
            if not root.right:
```



```

        return root.left

    cur = root.right
    while cur.left:
        cur = cur.left

    root.val = cur.val
    root.right = self.deleteNode(root.right, cur.val)

    return root

```

Range Sum Query - Mutable

```

class NumArray(object):

    def __init__(self, nums):
        self.n = len(nums)
        self.bit = [0] * (self.n + 1)
        self.arr = nums[:]

    def add(self, i, val):
        while i <= self.n:
            self.bit[i] += val
            i += i & -i

    for i, v in enumerate(nums):
        add(i + 1, v)

    def update(self, index, val):
        diff = val - self.arr[index]
        self.arr[index] = val

        i = index + 1
        while i <= self.n:
            self.bit[i] += diff
            i += i & -i

    def sumRange(self, left, right):
        def psum(i):
            s = 0
            while i > 0:
                s += self.bit[i]
                i -= i & -i
            return s

        return psum(right + 1) - psum(left)

```

Find Building Where Alice and Bob Can Meet

```
class Solution(object):
    def leftmostBuildingQueries(self, heights, queries):
        import bisect

        n = len(heights)
        # Normalize queries so  $a \leq b$ 
        indexed = []
        for i, (a, b) in enumerate(queries):
            if a > b:
                a, b = b, a
            indexed.append((b, a, i))
        indexed.sort(reverse=True) # sort by b descending

        # Sorted unique heights for compression
        hs = sorted(set(heights))
        m = len(hs)
        # Fenwick tree / BIT storing minimum index for each height rank
        INF = float('inf')
        bit = [INF] * (m + 1)

        def bit_update(i, val):
            while i <= m:
                if val < bit[i]:
                    bit[i] = val
                i += i & -i

        def bit_query(i):
            res = INF
            while i > 0:
                if bit[i] < res:
                    res = bit[i]
                i -= i & -i
            return res if res < INF else -1

        # Process buildings from the end
        res = [-1] * len(queries)
        j = n - 1
        for b, a, qi in indexed:
            # Add buildings with index > b to BIT
            while j > b:
                h = heights[j]
                # rank = position in sorted hs + 1, we want larger heights → lower index in BIT
                rank = m - bisect.bisect_left(hs, h)
                bit_update(rank, j)
                j -= 1

            if a == b or heights[a] < heights[b]:
                # They can meet at b
                res[qi] = b
            else:
                # Need a building j > b such that heights[j] > heights[a]
```

```

        # find all heights > heights[a] i.e. ranks ≤ threshold
        rank_thr = m - bisect.bisect_left(hs, heights[a])
        res[qi] = bit_query(rank_thr - 1)
    return res

```

My Calendar II

```
class MyCalendarTwo(object):
```

```

    def __init__(self):
        self.booked = []
        self.overlaps = []

    def book(self, start, end):
        # If this interval overlaps with an already double-booked interval → reject
        for s, e in self.overlaps:
            if not (end <= s or start >= e):
                return False

        # Record new overlaps with all single-booked intervals
        for s, e in self.booked:
            if not (end <= s or start >= e):
                self.overlaps.append((max(start, s), min(end, e)))

        # Insert into booked list
        self.booked.append((start, end))
        return True

```

Longest Common Prefix

```

class Solution(object):
    def longestCommonPrefix(self, strs):
        ans = ""
        for i in range(len(strs[0])):
            ch = strs[0][i]
            for s in strs:
                if i >= len(s) or s[i] != ch:
                    return ans
            ans += ch
        return ans

```

K-th Smallest in Lexicographical Order

```

class Solution(object):
    def findKthNumber(self, n, k):

        def count_steps(n, a, b):
            steps = 0
            while a <= n:
                steps += min(n + 1, b) - a
                a *= 10
                b *= 10
            return steps

        cur = 1

```

```
k -= 1
```

```
while k > 0:
```

```
    steps = count_steps(n, cur, cur + 1)
```

```
    if steps <= k:
```

```
        k -= steps
```

```
        cur += 1
```

```
    else:
```

```
        cur *= 10
```

```
        k -= 1
```

```
return cur
```

Longest Word in Dictionary

```
class Solution(object):
```

```
    def longestWord(self, words):
```

```
        words.sort()
```

```
        good = set([""])
```

```
        ans = ""
```

```
        for w in words:
```

```
            if w[:-1] in good:
```

```
                good.add(w)
```

```
                if len(w) > len(ans):
```

```
                    ans = w
```

```
        return ans
```

Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

```
class Solution(object):
```

```
    def findCriticalAndPseudoCriticalEdges(self, n, edges):
```

```
        # Add original index
```

```
        for i in range(len(edges)):
```

```
            edges[i].append(i)
```

```
        edges.sort(key=lambda x: x[2])
```

```
        # DSU
```

```
        def find(p, x):
```

```
            if p[x] != x:
```

```
                p[x] = find(p, p[x])
```

```
            return p[x]
```

```
        def union(p, r, a, b):
```

```
            pa = find(p, a)
```

```
            pb = find(p, b)
```

```
            if pa == pb:
```

```
                return False
```

```
            if r[pa] < r[pb]:
```

```
                pa, pb = pb, pa
```

```
            p[pb] = pa
```

```
            if r[pa] == r[pb]:
```

```

        r[pa] += 1
    return True

# MST weight with optional forced edge and optional blocked edge
def kruskal(blocked, forced):
    p = list(range(n))
    r = [0] * n
    weight = 0
    cnt = 0

    if forced != -1:
        a, b, w, _ = edges[forced]
        if union(p, r, a, b):
            weight += w
            cnt += 1

    for i, (a, b, w, _) in enumerate(edges):
        if i == blocked:
            continue
        if union(p, r, a, b):
            weight += w
            cnt += 1
        if cnt == n - 1:
            break

    return weight if cnt == n - 1 else float("inf")

# Get MST base weight
base = kruskal(-1, -1)

critical = []
pseudo = []

for i in range(len(edges)):
    w1 = kruskal(i, -1)
    if w1 > base:
        critical.append(edges[i][3])
    else:
        w2 = kruskal(-1, i)
        if w2 == base:
            pseudo.append(edges[i][3])

return [critical, pseudo]

```

Min Cost to Connect All Points

```

class Solution(object):
    def minCostConnectPoints(self, points):
        import heapq

        n = len(points)
        used = [False] * n
        heap = [(0, 0)] # (cost, point)

```

```

ans = 0
taken = 0

while taken < n:
    cost, u = heapq.heappop(heap)
    if used[u]:
        continue
    used[u] = True
    ans += cost
    taken += 1

    x1, y1 = points[u]
    for v in range(n):
        if not used[v]:
            x2, y2 = points[v]
            dist = abs(x1 - x2) + abs(y1 - y2)
            heapq.heappush(heap, (dist, v))

return ans

```

Lemonade Change

```

class Solution(object):
    def lemonadeChange(self, bills):
        five = 0
        ten = 0

        for b in bills:
            if b == 5:
                five += 1
            elif b == 10:
                if five == 0:
                    return False
                five -= 1
                ten += 1
            else: # b == 20
                if ten > 0 and five > 0:
                    ten -= 1
                    five -= 1
                elif five >= 3:
                    five -= 3
                else:
                    return False

        return True

```

Candy

```

class Solution(object):
    def candy(self, ratings):
        n = len(ratings)
        left = [1] * n
        right = [1] * n

```

```

for i in range(1, n):
    if ratings[i] > ratings[i - 1]:
        left[i] = left[i - 1] + 1

for i in range(n - 2, -1, -1):
    if ratings[i] > ratings[i + 1]:
        right[i] = right[i + 1] + 1

ans = 0
for i in range(n):
    ans += max(left[i], right[i])

return ans

```

Container With Most Water

```

class Solution(object):
    def maxArea(self, height):
        l, r = 0, len(height) - 1
        ans = 0

        while l < r:
            ans = max(ans, (r - l) * min(height[l], height[r]))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1

        return ans

```

Climbing Stairs

```

class Solution(object):
    def climbStairs(self, n):
        if n <= 2:
            return n

        a, b = 1, 2
        for _ in range(3, n + 1):
            a, b = b, a + b

        return b

```

House Robber

```

class Solution(object):
    def rob(self, nums):
        prev2 = 0
        prev1 = 0

        for x in nums:
            prev2, prev1 = prev1, max(prev1, prev2 + x)

        return prev1

```

Target Sum

```

class Solution(object):

```

```

def findTargetSumWays(self, nums, target):
    from collections import defaultdict

    dp = defaultdict(int)
    dp[0] = 1

    for x in nums:
        ndp = defaultdict(int)
        for s in dp:
            ndp[s + x] += dp[s]
            ndp[s - x] += dp[s]
        dp = ndp

    return dp[target]

```

K-Concatenation Maximum Sum

```

class Solution(object):
    def kConcatenationMaxSum(self, arr, k):
        mod = 10**9 + 7

        def kadane(a):
            best = 0
            cur = 0
            for x in a:
                cur = max(0, cur + x)
                best = max(best, cur)
            return best

        if k == 1:
            return kadane(arr) % mod

        total = sum(arr)
        best_two = kadane(arr + arr)

        if total > 0:
            return (best_two + (k - 2) * total) % mod
        else:
            return best_two % mod

```

Interleaving String

```

class Solution(object):
    def isInterleave(self, s1, s2, s3):
        if len(s1) + len(s2) != len(s3):
            return False

        m, n = len(s1), len(s2)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        for i in range(m + 1):
            for j in range(n + 1):
                if i > 0 and s1[i - 1] == s3[i + j - 1] and dp[i - 1][j]:
                    dp[i][j] = True
                if j > 0 and s2[j - 1] == s3[i + j - 1] and dp[i][j - 1]:
                    dp[i][j] = True

        return dp[m][n]

```

Word Break

```

class Solution(object):

```



```

def wordBreak(self, s, wordDict):
    wordSet = set(wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True
                break

    return dp[n]

```

Find Greatest Common Divisor of Array

```

class Solution(object):
    def findGCD(self, nums):

        def gcd(a, b):
            while b:
                a, b = b, a % b
            return a

        mn = min(nums)
        mx = max(nums)
        return gcd(mn, mx)

```

The k-th Factor of n

```

class Solution(object):
    def kthFactor(self, n, k):
        factors = []
        i = 1

        while i * i <= n:
            if n % i == 0:
                factors.append(i)
                if i != n // i:
                    factors.append(n // i)
            i += 1

        factors.sort()
        return factors[k - 1] if k <= len(factors) else -1

```

Maximum Prime Difference

```

class Solution(object):
    def maximumPrimeDifference(self, nums):

        def isPrime(x):
            if x < 2:
                return False
            i = 2
            while i * i <= x:
                if x % i == 0:
                    return False
                i += 1
            return True

```

```

        return True

    first = -1
    last = -1

    for i in range(len(nums)):
        if isPrime(nums[i]):
            first = i
            break

    for i in range(len(nums)-1, -1, -1):
        if isPrime(nums[i]):
            last = i
            break

    return last - first

```

Nim Game

```

class Solution(object):
    def canWinNim(self, n):
        return n % 4 != 0

```

Cat and Mouse

```

class Solution(object):
    def catMouseGame(self, graph):
        n = len(graph)

        # dp[m][c][t] → result when mouse=m, cat=c, turn=t
        # t = 0 → mouse turn
        # t = 1 → cat turn
        dp = [[[0] * 2 for _ in range(n)] for _ in range(n)]

        from collections import deque
        q = deque()

        # Base cases
        for i in range(n):
            for t in range(2):
                dp[0][i][t] = 1 # mouse reaches hole → mouse wins
                q.append((0, i, t, 1))

            if i != 0:
                dp[i][i][t] = 2 # cat catches mouse
                q.append((i, i, t, 2))

        # degree count
        deg = [[[0] * 2 for _ in range(n)] for _ in range(n)]
        for m in range(n):
            for c in range(n):
                deg[m][c][0] = len(graph[m]) # mouse moves
                deg[m][c][1] = len([x for x in graph[c] if x != 0]) # cat moves (can't go to hole)

        # BFS backward
        while q:

```

```

m, c, t, res = q.popleft()

# predecessor states
if t == 0:
    # previous move was cat
    for pc in [x for x in graph[c] if x != 0]:
        pt = 1
        if dp[m][pc][pt] == 0:
            if res == 2: # cat winning child → cat wins
                dp[m][pc][pt] = 2
                q.append((m, pc, pt, 2))
            else:
                deg[m][pc][pt] -= 1
                if deg[m][pc][pt] == 0:
                    dp[m][pc][pt] = 1
                    q.append((m, pc, pt, 1))
        else:
            # previous move was mouse
            for pm in graph[m]:
                pt = 0
                if dp[pm][c][pt] == 0:
                    if res == 1: # mouse winning child → mouse wins
                        dp[pm][c][pt] = 1
                        q.append((pm, c, pt, 1))
                    else:
                        deg[pm][c][pt] -= 1
                        if deg[pm][c][pt] == 0:
                            dp[pm][c][pt] = 2
                            q.append((pm, c, pt, 2))

return dp[1][2][0]

```